

Admin_Views_Popular_Equipment.sql Explanation

SQL Code

```
--* =====
--* POPULAR EQUIPMENT VIEW
--* =====
--* DBMS Concept: Aggregation with JOINS
--* Purpose: Show equipment popularity based on usage in training records
--* =====

CREATE OR REPLACE VIEW popular_equipment AS
SELECT
    ae.equipment_id,
    ae.equipment_name,
    ae.equipment_type,
    --* Count exercises using this equipment
    COUNT(DISTINCT e.exercise_id) as exercise_count,
    --* Count total workouts using this equipment
    COUNT(tr.record_id) as total_workouts,
    --* Count unique users who used this equipment
    COUNT(DISTINCT tr.user_id) as unique_users,
    --* Calculate total reps performed with this equipment
    COALESCE(SUM(tr.sets * tr.reps), 0) as total_reps,
    --* Calculate average weight used
    ROUND(AVG(tr.weight_used), 2) as avg_weight_used,
    --* Calculate total volume (weight * sets)
    COALESCE(SUM(tr.weight_used * tr.sets), 0) as total_volume,
    --* Find first and last usage dates
    MIN(tr.date) as first_used_date,
    MAX(tr.date) as last_used_date
FROM available equipments ae
LEFT JOIN exercises e ON ae.equipment_id = e.equipment_id
LEFT JOIN training_records tr ON e.exercise_id = tr.exercise_id
GROUP BY ae.equipment_id, ae.equipment_name, ae.equipment_type
HAVING COUNT(tr.record_id) > 0
ORDER BY total_workouts DESC, exercise_count DESC, ae.equipment_name;
```

Explanation (Line-by-line)

This section now explains each part of the SQL view **line-by-line**, exactly like you requested for the previous file.

```
CREATE OR REPLACE VIEW popular_equipment AS
```

- Creates a view named **popular_equipment**.
- `OR REPLACE` makes sure that if the view already exists, it will be overwritten.

```
SELECT
```

- Starts selecting all the columns that will appear inside the view.

```
ae.equipment_id
```

- `ae` is the alias for the table **available equipments**.
- This line selects the equipment's unique ID.

```
ae.equipment_name
```

- Selects the readable name of the equipment.

```
ae.equipment_type
```

- Shows what type of equipment it is (dumbbell, barbell, machine, etc.).

```
COUNT(DISTINCT e.exercise_id) as exercise_count
```

- Counts how many **different exercises** use this equipment.
- `DISTINCT` ensures no duplicates.
- This tells how versatile the equipment is.

```
COUNT(tr.record_id) as total_workouts
```

- Counts how many workout records used this equipment.
- Every time a user performed an exercise with this equipment, it adds to the count.

```
COUNT(DISTINCT tr.user_id) as unique_users
```

- Counts how many **different users** have used this equipment.
- Helps understand how widely the equipment is used across users.

```
COALESCE(SUM(tr.sets * tr.reps), 0) as total_reps
```

- Calculates **total reps** performed using this equipment.
- `SUM(tr.sets * tr.reps)` multiplies sets × reps for every row.
- `COALESCE(..., 0)` ensures the value is `0` instead of `NULL` if no workouts exist.

```
ROUND(AVG(tr.weight_used), 2) as avg_weight_used
```

- Takes the **average weight** used with this equipment.
- Rounded to 2 decimal places.

```
COALESCE(SUM(tr.weight_used * tr.sets), 0) as total_volume
```

- Calculates **total volume lifted** using this equipment.
- Volume = weight × sets.
- `COALESCE` again makes sure NULL becomes 0.

```
MIN(tr.date) as first_used_date
```

- Finds the **earliest date** this equipment was used.

```
MAX(tr.date) as last_used_date
```

- Finds the **latest date** this equipment was used.
-

FROM and JOIN section

```
FROM available equipments ae
```

- This is the **main/base table**.
- `ae` is an alias to make referencing easier.

```
LEFT JOIN exercises e ON ae.equipment_id = e.equipment_id
```

- Connects equipment to exercises that use it.
- `LEFT JOIN` ensures equipment appears **even if no exercise uses it**.

```
LEFT JOIN training_records tr ON e.exercise_id = tr.exercise_id
```

- Connects exercises to actual workout records.
 - Another `LEFT JOIN` ensures we show equipment even if workouts are missing.
-

GROUPING, FILTERING, ORDERING

```
GROUP BY ae.equipment_id, ae.equipment_name, ae.equipment_type
```

- Groups results by each equipment item.
- All aggregation functions (COUNT, SUM, AVG, MIN, MAX) work **within each group**.

```
HAVING COUNT(tr.record_id) > 0
```

- `HAVING` filters after grouping.
- Ensures only equipment **that was actually used at least once** appears.
- Unlike WHERE, HAVING can filter on aggregated values.

```
ORDER BY total_workouts DESC, exercise_count DESC, ae.equipment_name
```

- Sorts equipment by:
 - Most used first (`total_workouts DESC`)
 - If tie → equipment used in more exercises first
 - If still tie → alphabetical by name
-

Summary (easy to say in viva)

- This view shows which gym equipment is most popular.
- It uses `LEFT JOIN` to combine equipment, exercises, and training records.
- Uses aggregation functions like COUNT, SUM, AVG, MIN, MAX.
- `COALESCE` avoids NULL by replacing it with 0.
- `GROUP BY` organizes data per equipment.
- `HAVING` ensures only equipment that was actually used appears.
- Sorted so the most used equipment appears at the top.